

1. С помощью каких команд SQL можно найти самую длинную строчку в столбце?

```
SELECT column_name FROM table_name ORDER BY LENGTH(column_name) DESC LIMIT 1;
```

2. Как узнать имя docker контейнера, если нам известен host и port сервиса поднятого в нем? Как посмотреть его логи?

Если у нас есть доступ к хосту, то выполняем команду:

```
docker ps -a --filter "publish=<port>"
```

В выводе будет имя контейнера.

```
docker logs <container_name>
```

посмотреть его логи

3. По какой причине может возникать ошибка 404?

Расскажите как вы будете ее анализировать

Ошибка 404 (Not Found) возникает, когда клиент пытается получить доступ к ресурсу, который не существует или недоступен на сервере. Это может быть вызвано различными причинами, включая:

1. **Неправильный URL:** Клиент ввел неправильный или устаревший URL.
2. **Перемещение или удаление ресурса:** Файл или страница были перемещены или удалены без соответствующего перенаправления.
3. **Проблемы с маршрутизацией:** На сервере неправильно настроена маршрутизация запросов.
4. **Проблемы с конфигурацией сервера:** Неправильная конфигурация сервера или веб-сервера (например, Apache, Nginx).
5. **Проблемы с DNS:** Неправильная настройка DNS, которая может приводить к неправильному разрешению доменных имен.

Анализ ошибки 404

Для анализа ошибки 404 можно выполнить следующие шаги:

1. **Проверка URL:** Убедитесь, что URL, который вы пытаетесь открыть, правильный и актуальный.

2. Проверка перенаправлений: Если вы перемещали или удаляли страницы, убедитесь, что настроены правильные перенаправления (редиректы) на новые URL.

3. Проверка логов сервера: Посмотрите логи сервера (например, Apache, Nginx) для получения дополнительной информации о запросах, которые приводят к ошибке 404. Логи могут содержать информацию о том, какие файлы или ресурсы не найдены.

4. Проверка конфигурации сервера: Убедитесь, что конфигурация сервера правильная и что все необходимые директории и файлы доступны. Например, проверьте файлы конфигурации веб-сервера (например, `httpd.conf` для Apache или `nginx.conf` для Nginx).

5. Проверка DNS: Если проблема возникает с определенным доменом, проверьте настройки DNS, чтобы убедиться, что домен правильно разрешается на IP-адрес вашего сервера.

6. Проверка файловой системы: Убедитесь, что файлы и директории, к которым вы пытаетесь получить доступ, существуют на сервере и что у веб-сервера есть необходимые права доступа к этим файлам.

7. Проверка маршрутизации: Если вы используете фреймворк или CMS, убедитесь, что маршрутизация запросов настроена правильно.

8. Тестирование с других устройств и сетей: Иногда проблема может быть специфичной для конкретного устройства или сети. Попробуйте открыть URL с другого устройства или из другой сети, чтобы исключить эту возможность.

9. Проверка зависимостей: Если ваш сервис зависит от других сервисов или API, убедитесь, что они доступны и работают корректно.

Выполнение этих шагов поможет вам определить причину ошибки 404 и предпринять соответствующие действия для ее устранения.

4. Какие способы авторизации в методах api вам знакомы?

Авторизация в методах API — это процесс проверки и подтверждения прав пользователя на доступ к определенным ресурсам или выполнение определенных действий.

Существует несколько широко используемых методов авторизации в API:

1. API Key:

- Описание: Простой ключ, который передается в заголовке или параметре запроса.
- Пример: `Authorization: ApiKey <your_api_key>`

2. Basic Authentication:

- Описание: Передача имени пользователя и пароля в заголовке запроса в закодированном виде (Base64).

- Пример: `Authorization: Basic dXNlcm5hbWU6cGFzc3dvcmQ=``

3. Bearer Token:

- Описание: Токен, который передается в заголовке запроса и обычно используется для OAuth 2.0.

- Пример: `Authorization: Bearer <your\_access\_token>`

4. OAuth 2.0:

- Описание: Протокол авторизации, который позволяет сервисам получать ограниченный доступ к аккаунту пользователя без передачи данных для входа.

- Пример: Использование access token для доступа к защищенным ресурсам.

5. JWT (JSON Web Token):

- Описание: Компактный формат для передачи информации между сторонами в виде JSON объекта. JWT часто используется для авторизации.

- Пример: `Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...`

6. Digest Authentication:

- Описание: Улучшенная версия Basic Authentication, которая использует хеширование пароля для большей безопасности.

- Пример: Использование хешированного значения пароля в заголовке запроса.

7. HMAC (Hash-based Message Authentication Code):

- Описание: Использование секретного ключа и хеш-функции для создания кода аутентификации сообщения.

- Пример: `Authorization: HMAC <hashed\_value>`

8. OpenID Connect:

- Описание: Протокол на основе OAuth 2.0, который добавляет функциональность для аутентификации пользователей.

- Пример: Использование ID Token для аутентификации пользователя.

9. SAML (Security Assertion Markup Language):

- Описание: Формат для обмена аутентификационной и авторизационной информацией между сторонами, часто используемый в корпоративных средах.

- Пример: Использование SAML assertions для авторизации.

10. Custom Authentication Schemes:

- Описание: Разработка собственных методов авторизации, которые могут включать комбинации вышеупомянутых методов или полностью новые подходы.

- Пример: Использование собственного протокола или формата токенов.

Каждый из этих методов имеет свои преимущества и недостатки, и выбор конкретного метода зависит от требований безопасности, сложности системы и других факторов.

5. Напишите тесткейсы для проверки следующего метода:

пример запроса:

http://test:8090/createPhoneNumber пример тела запроса: {
"phoneNumber": "9998753322",
"code": "7",
"dateFrom": "2024-05-15"
"dateTo": "2024-10-15"
}

Тест-кейсы для проверки метода `createPhoneNumber` должны охватывать различные аспекты, включая валидацию входных данных, обработку ошибок и корректность создания записи. Вот примеры тест-кейсов:

Тест-кейс 1: Успешное создание записи

Описание: Проверка успешного создания записи с корректными данными.

- URL: `http://test:8090/createPhoneNumber`

- Метод: POST

- Тело запроса:

```
json
{
  "phoneNumber": "9998753322",
  "code": "7",
  "dateFrom": "2024-05-15",
  "dateTo": "2024-10-15"
}
```

- Ожидаемый результат:

- Статус-код: 200 OK

- Тело ответа: Сообщение о успешном создании записи.

Тест-кейс 2: Некорректный формат номера телефона

Описание: Проверка обработки некорректного формата номера телефона.

- URL: `http://test:8090/createPhoneNumber`

- Метод: POST

- Тело запроса:

```
json
{
  "phoneNumber": "999875332",
  "code": "7",
  "dateFrom": "2024-05-15",
```

```
    "dateTo": "2024-10-15"
  }
```

- Ожидаемый результат:
- Статус-код: 400 Bad Request
- Тело ответа: Сообщение об ошибке, указывающее на некорректный формат номера телефона.

Тест-кейс 3: Некорректный формат даты

Описание: Проверка обработки некорректного формата даты.

- URL: `http://test:8090/createPhoneNumber`
- Метод: POST
- Тело запроса:

```
json
{
    "phoneNumber": "9998753322",
    "code": "7",
    "dateFrom": "2024-15-05",
    "dateTo": "2024-10-15"
}
```

- Ожидаемый результат:
- Статус-код: 400 Bad Request
- Тело ответа: Сообщение об ошибке, указывающее на некорректный формат даты.

Тест-кейс 4: Отсутствие обязательного поля

Описание: Проверка обработки отсутствия обязательного поля.

- URL: `http://test:8090/createPhoneNumber`
- Метод: POST
- Тело запроса:

```
json
{
    "phoneNumber": "9998753322",
    "code": "7",
    "dateTo": "2024-10-15"
}
```

- Ожидаемый результат:
- Статус-код: 400 Bad Request
- Тело ответа: Сообщение об ошибке, указывающее на отсутствие обязательного поля `dateFrom`.

Тест-кейс 5: Некорректный код

Описание: Проверка обработки некорректного кода.

- URL: `http://test:8090/createPhoneNumber`

- Метод: POST

- Тело запроса:

json

```
{
    "phoneNumber": "9998753322",
    "code": "10",
    "dateFrom": "2024-05-15",
    "dateTo": "2024-10-15"
}
```

- Ожидаемый результат:

- Статус-код: 400 Bad Request

- Тело ответа: Сообщение об ошибке, указывающее на некорректный код.

Тест-кейс 6: Дата "dateFrom" больше даты "dateTo"

Описание: Проверка обработки случая, когда дата "dateFrom" больше даты "dateTo".

- URL: `http://test:8090/createPhoneNumber`

- Метод: POST

- Тело запроса:

json

```
{
    "phoneNumber": "9998753322",
    "code": "7",
    "dateFrom": "2024-10-15",
    "dateTo": "2024-05-15"
}
```

- Ожидаемый результат:

- Статус-код: 400 Bad Request

- Тело ответа: Сообщение об ошибке, указывающее на некорректное соотношение дат.

Тест-кейс 7: Дублирование номера телефона

Описание: Проверка обработки случая, когда номер телефона уже существует.

- URL: `http://test:8090/createPhoneNumber`

- Метод: POST

- Тело запроса:

json

```
{
    "phoneNumber": "9998753322",
    "code": "7",
    "dateFrom": "2024-05-15",
    "dateTo": "2024-10-15"
}
```

- Ожидаемый результат:
- Статус-код: 409 Conflict
- Тело ответа: Сообщение об ошибке, указывающее на дублирование номера телефона.

Эти тест-кейсы помогут проверить различные аспекты метода ``createPhoneNumber`` и убедиться, что он корректно обрабатывает как валидные, так и невалидные запросы.

6. Опишите своими словами, чем отличаются критический и блокирующий баг.

Критический и блокирующий баги — это два термина, которые используются в области тестирования программного обеспечения для описания серьезности дефектов. Однако, между ними есть некоторые различия:

Критический баг

это дефект, который приводит к серьезным последствиям и значительно влияет на функциональность или производительность системы. Критические баги обычно приводят к сбоям в работе основных функций системы, но не обязательно делают ее полностью непригодной для использования. Например, критический баг может вызвать падение системы, неправильное отображение данных или невозможность выполнения ключевых операций.

Блокирующий баг

это дефект, который полностью препятствует выполнению каких-либо операций или использованию системы в целом. Блокирующие баги делают систему неработоспособной и не позволяют пользователям или тестировщикам продолжать работу. Например, блокирующий баг может привести к тому, что система не запускается, не отвечает на запросы или не позволяет выполнить критически важные задачи.

Основные отличия

- Степень влияния: Критический баг может серьезно повлиять на функциональность, но не обязательно делает систему полностью непригодной. Блокирующий баг полностью препятствует использованию системы.
- Возможность работы: При наличии критического бага пользователи могут продолжать использовать систему, но с ограничениями. Блокирующий баг делает систему неработоспособной, и пользователи не могут продолжать работу.
- Приоритет исправления: Оба типа багов имеют высокий приоритет для исправления, но блокирующие баги обычно требуют более срочного вмешательства, так как они полностью блокируют работу.

В целом, оба типа багов требуют быстрого и эффективного решения, но блокирующие баги имеют более высокий уровень срочности и критичности для работы системы.

7. Как вы понимаете интеграционное тестирование?

Интеграционное тестирование (Integration Testing) — это один из видов тестирования программного обеспечения, который направлен на проверку взаимодействия между различными компонентами, модулями или сервисами системы. Цель интеграционного тестирования — выявить дефекты, связанные с коммуникацией и обменом данными между этими компонентами.